

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«МИРЭА – Российский технологический университет»**
**РТУ МИРЭА**

**Колледж программирования и кибербезопасности**

**УЧЕБНАЯ ДИСЦИПЛИНА/**
**МЕЖДИСЦИПЛИНАРНЫЙ КУРС**
**МДК.01.04 Системное программирование**

**ПЦК Программирования**

**ВИД УЧЕБНОГО**
**МАТЕРИАЛА Лекционные материалы**

**ПРЕПОДАВАТЕЛЬ Иванова Дарья Андреевна**

# 4. Интерфейсы Файловой системы

Эта лекция описывает функции библиотеки GNU C для управления файлами. В отличие от функций ввода и функций вывода, эти функции имеют отношение к действиям на файлах непосредственно.

Среди средств, описанных в этой главе - функции для исследования или изменения каталогов, функции для переименования и удаления файлов, и функции для исследования и установки атрибутов файла типа прав доступа и изменения времени.

# 4.1 Рабочий каталог

Каждый процесс связанный с этим каталогом, называемым текущим рабочим каталогом или просто рабочим каталогом, который используется в уточнении имен файла прямого доступа (см. Раздел 6.2.2 [Уточнение Имени файла]).

Когда Вы регистрируетесь в системе и начинаете новый сеанс, ваш рабочий каталог первоначально устанавливается в исходный каталог, связанный с вашим логическим входом в систему в базе данных пользователей системы. Вы можете находить исходный каталог любого пользователя, используются getpwuid или getpwnam функции; см. Раздел 25.12 [База данных Пользователей].

Пользователи могут изменять рабочий каталог, используя команды оболочки подобно cd. Функции, описанные в этом разделе - примитивы, используемые командами и другими программами для исследования и изменения рабочего каталога.

Прототипы для этих функций объявлены в файле 'unistd.h'.

```
char * getcwd (char *buffer, size_t size)  (функция)
```

Getcwd функция возвращает абсолютное имя файла, представляющее текущий рабочий каталог, сохраняя его в символьном буфере, который Вы указываете. Аргумент size - то, как Вы сообщаете системе размер резервирования буфера.

Версия GNU этой функции также разрешает Вам определять пустой указатель для буферного аргумента. Тогда getcwd зарезервирует буфер автоматически, как malloc (см. Раздел 3.3 [Беспрепятственное Резервирование]).

Возвращаемое значение - буфер в случае успеха и пустой указатель при отказе. Следующие errno условия ошибки определены для этой функции:

**EINVAL**

аргумент size - нуль, и буфер - не пустой указатель.

**ERANGE**

аргумент size - меньше чем длина имени рабочего каталога.

Вы должны зарезервировать больший массив и попытаться снова.

**EACCES**

Права на чтение или поиск компонент имени файла были превышены.

Вот пример, показывающий, как Вы могли бы реализовать GNU getcwd (NULL, 0) используя только стандартное поведение getcwd:

```
char *
gnu_getcwd ()
{
        int size = 100;
        char *buffer = (char *) xmalloc (size);
        while (1)
                {
                        char*value = getcwd(buffer,size);
                        if (value != 0)
                                return buffer;
                        size *= 2;
                        free (buffer);
                        buffer = (char *) xmalloc (size);
                }
}
```

См. Раздел 3.3.2 [Примеры Malloc], для уточнения информации относительно xmalloc, которая - не библиотечная функция, но обычное имя, используемое в большинстве программного обеспечения GNU.

```
char * getwd (char *buffer)  (функция)
```

Подобна getcwd. Библиотека GNU обеспечивает getwd для совместимости в обратном направлении с BSD. Буфер должен быть указатель на массив по крайней мере PATH_MAX байтов.

```
-Функция:    int chdir (const char *filename)
```

Эта функция используется, чтобы установить рабочий каталог процесса filename.

Нормальное, успешное возвращаемое значение из chdir - 0. Значение -1 возвращается, чтобы указать ошибку. Errno условия ошибки, определенные для этой функции - обычные синтаксические ошибки имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), плюс ENOTDIR, если filename файла - не каталог.

# 4.2 Доступ в Каталоги

Средства, описанные в этом разделе допускают Вам читать содержимое файла каталога. Это полезно, если Вы хотите, чтобы ваша программа перечислила все файлы в каталоге, возможно как часть меню.

Opendir функция открывает поток каталога, чьи элементы являются входами каталога. Вы используете readdir функцию на потоке каталога, чтобы отыскать эти входы, представляемые как объекты struct dirent. Имя файла для каждого входа сохранено в d_name элементе этой структуры. Имеются очевидные параллели здесь со средствами потока для обычных файлов, описанных в Главе 7 [ввод вывод на Потоках].

## Формат Входа в Каталог

Этот раздел описывает то, что Вы находите входя в каталога, поскольку Вы могли бы получать это из потока каталога. Все символы объявлены в заглавном файле 'dirent.h'.

### -Тип данных: struct dirent

Это - тип структуры, используемый, чтобы возвратить информацию относительно входов в каталог. Она содержит следующие поля:

```
char *d_name
```

Это - компонента имени файла с нулевым символом в конце. Это  единственное поле, на которое Вы можете рассчитывать во всех POSIX системах.

```
ino_t d_fileno
```

Это - серийный номер файла. Для BSD совместимости, Вы можете также обратиться к этому элементу как к d_ino.

```
size_t d_namlen
```

Это - длина имени файла, исключая пустой символ завершения.

Эта структура в будущем может содержать дополнительные элементы.

Когда файл имеет многочисленные имена, каждое имя имеет собственный вход в каталог. Единственный способ которым Вы можете сообщать, что входы каталога принадлежат одиночному файлу - это, если они имеют то же самое значение для d_fileno поля.

Атрибуты Файла типа размера, числа изменений, и т.п. - часть файла непосредственно, см. Раздел 9.8 [Атрибуты Файла].

## Открытие Потока Каталога

Этот раздел описывает, как открыть поток каталога. Все символы объявлены в заглавном файле 'dirent.h'.

### -Тип данных DIR

Тип данных DIR представляет поток каталога.

Вы не должны когда-либо резервировать объекты struct dirent или типов данных DIR, так как функции доступа каталога делают это для Вас. Вместо этого, Вы обращаетесь к этим объектам, используя указатели, возвращенные следующими функциями.

```
DIR * opendir (const char *dirname)  (функция)
```

Opendir функция открывает и возвращает поток каталога для чтения каталога, чье имя dirname. Поток имеет тип DIR *.

Если происходит ошибка opendir возвращает пустой указатель. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Имя файла Errors]), следующие errno условия ошибки определены для этой функции:

**EACCES право чтения отклонено для каталога dirname.**

**EMFILE процесс имеет слишком много открытых файлов.**

**ENFILE вся система, или возможно файловая система, которая**

содержит каталог, не может поддерживать любые дополнительные открытые файлы в настоящее время. (Эта проблема не может случаться в системе GNU.)

Тип DIR обычно реализован, используя дескриптор файла, а opendir функция в терминах функции open. См. Главу 8 [ввод - вывод низкого уровня]. Потоки Каталога и основные дескрипторы файла закрываются при запуске (см. Раздел 23.5 [выполнение Файла]).

## Чтение и Закрытие Потока Каталога

Этот раздел описывает, как читать входы каталога из потока каталога, и как закрыть поток, когда Вы закончили работу с ним. Все символы объявлены в файле 'dirent.h'.

```
struct dirent * readdir (DIR *dirstream)  (функция)
```

Эта функция читает следующий вход из каталога. Она обычно возвращает указатель на структуру, содержащую информацию относительно файла. Эта структура статически размещена и может быть перезаписана последующим обращением.

Примечание Переносимости: На некоторых системах, readdir не может возвращать входы для '. ' и ' .. ', даже если они - всегда допустимые имена файла в любом каталоге. См. Раздел 6.2.2 [Уточнение Имени файла].

Если нет больше входов в каталоге, или обнаружена ошибка, readdir возвращает пустой указатель. Следующие errno условия ошибки определены для этой функции:

**EBADF**

irstream аргумент не допустим.

```
int closedir (DIR *dirstream)  (функция)
```

Эта функция закрывает поток каталога dirstream. Она возвращает 0 при успехе и -1 при отказе.

Следующие errno условия ошибки определены для этой функции:

**EBADF**

dirstream аргумент не допустим.

## Простая Программа Просмотра Каталога

Имеется простая программа, которая печатает имена файлов в текущем рабочем каталоге:

```
#include <stddef.h>
#include <stdio.h>
#include <sys/types.h>
#include <dirent.h>
int
main (void)
{
        DIR *dp;
        struct dirent *ep;
        dp = opendir ('./');
        if (dp != NULL)
                {
                        while (ep = readdir (dp))
                                puts (ep->d_name);
                        (void) closedir (dp);
                }
        else
                puts ('Couldn't open the directory.');
        return 0;
}
```

## Произвольный доступ в Потоке Каталога

Этот раздел описывает, как повторно читать части каталога, который Вы уже читали из открытого потока каталога. Все символы объявлены в заглавном файле 'dirent.h'.

```
void rewinddir (DIR *dirstream)  (функция)
```

Rewinddir функция используется, чтобы повторно инициализировать поток каталога dirstream, так, чтобы, если Вы вызвали readdir, она возвращала информацию относительно первого входа в каталоге снова.

```
off_t telldir (DIR *dirstream)  (функция)
```

Telldir функция возвращает файловую позицию потока каталога dirstream. Вы можете использовать это значение с seekdir, чтобы восстановить поток каталога в эту позицию.

```
void seekdir (DIR *dirstream, off_t pos) Function
```

Seekdir функция устанавливает файловую позицию потока каталога dirstream в pos.

Значение pos должно быть результатом предыдущего обращения к telldir на этом специфическом потоке; закрытие и повторное открытие каталога может объявить неверным значения, возвращенные telldir.

# 4.3 Жесткие Связи

В POSIX системах, один файл может иметь много имен в то же самое время. Все имена в равной степени реальны, и никакое из них не лучше других.

Чтобы добавить имя к файлу, используйте функцию link. (Новое имя также называется жесткой связью с файлом.) Создание нового обращения к файлу не копирует содержимое файла; просто появляется новое имя, под которым файл может быть известен, в дополнение к существующему имени файла (или именам).

Один файл может иметь имена в отдельных каталогах, так что, организация файловой системы - не строгая иерархия или дерево.

Так как специфический файл существует внутри одиночной файловой системы, все имена должны быть в каталогах в этой файловой системе. Link сообщает ошибку, если Вы пробуете делать жесткое обращение к файлу из другой файловой системы.

Прототип для функции link объявлен в заглавном файле 'unistd.h'.

```
int link (const char *oldname, const char *newname)  (функция)
```

Функция link делает новую ссылка с существующим файлом, называемым oldname, под новым именем newname.

Эта функция возвращает значение 0, если она успешна и -1 при отказе. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]) и для oldname и newname, следующие errno условия ошибки определены для этой функции:

**EACCES**

каталог, в котором link должна написать, нельзя перезаписывать.

**EEXIST**

имеется уже файл, именованный newname. Если Вы хотите заменить эту ссылку на новую ссылку, Вы должны сначала явно удалить старую ссылку.

**EMLINK**

имеются уже слишком много связей к файлу, именованному oldname. (Максимальное число связей к файлу - LINK_MAX; см. Раздел 27.6 [Ограничения для Файлов].)

Хорошо разработанные файловые системы никогда не сообщают эту ошибку, потому что они разрешают большее количество связей чем ваш диск мог бы содержать. Однако, Вы должны все еще принимать во внимание возможности этой ошибки, поскольку она могла бы следовать из доступа к сети к файловой системе на другой машине.

**ENOENT**

файл, именованный oldname не существует. Вы не можете делать ссылок к файлу, который не существует.

**ENOSPC**

каталог или файловая система, которая содержала бы, новую ссылка 'полна' и не может быть расширена.

**EPERM**

Некоторые реализации позволяют только привилегированным пользователям делать связи к каталогам, а другие запрещают эту операцию полностью. Эта ошибка используется, чтобы сообщить эту проблему.

**EROFS**

Каталог, содержащий новую ссылка не может изменяться, потому что он находится в файловой системе только для чтения.

**EXDEV**

Каталог, заданный в newname находится в другой файловой системе чем существующий файл.

# 4.4 Символические Связи

Система GNU поддерживает связи программного обеспечения или символические связи. Это - вид 'файла', который является по существу указателем на другое имя файла. В отличие от жестких обращений, символические связи могут быть сделаны к каталогам или между файловыми системами без ограничений.

Функция open понимает, что Вы передали имя ссылки, и читает имя файла, вместо этого открывает файл, указываемый ссылкой. Stat функция аналогично функционирует на файле, на который указывает символическая ссылка, а не на связи непосредственно. Как связывается, функция, которая делает жесткое обращение.

Наоборот, другие операции типа удаления или переименования файла воздействуют непосредственно на ссылки. Функции readlink и lstat также воздерживаются от следующих символических ссылок, потому что их цель - получить информацию относительно ссылки.

Прототипы для функций, перечисленных в этом разделе находятся в 'unistd.h'.

**-Функция: int symlink (const char *oldname, const char *newname)**

Symlink функция делает символическую ссылку к oldname, с именем newname.

Нормальное возвращаемое значение из symlink - 0. Возвращаемое значение -1 указывает ошибку.

В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:

**EEXIST**

имеется уже существующий файл по имени newname.

**EROFS**

файл newname существовал бы на файловой системе только для чтения.

**ENOSPC**

каталог или файловая система не может быть расширена, чтобы делать новую ссылка.

**EIO**

аппаратная ошибка произошла при чтении или записи данных относительно диска.

## -Функция: int readlink (const char *filename, char *buffer, size_t size)

Readlink функция получает значение символической filename связи. Имя файла, на которое ссылка указывает, скопировано в буфер. Эта строка имени файла не с нулевым символом в конце; readlink обычно возвращает скопированное число символов. Аргумент size определяет максимальное число символов, для копирования, обычно размер резервирования буфера.

Если возвращаемое значение равняется size, Вы не можете распознать, имелся или нет участок памяти, чтобы возвратить все имя. Так что делайте больший буфер, и вызовите readlink снова. Вот пример:

```
char *
readlink_malloc (char *filename)
{
        int size = 100;
        while (1)
        {
                char *buffer = (char *) xmalloc (size);
                int nchars= readlink(filename,buffer,size);
                if (nchars < size)
                return buffer;
                free (buffer);
                size *= 2;
        }
}
```

Значение -1 возвращается в случае ошибки. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:
**EINVAL**

именованный файл - не символическая ссылка.

**EIO**

аппаратная ошибка произошла при чтении или записи данных относительно диска.

# 4.5 Удаление Файлов

Вы можете удалять файл функциями unlink или remove. (Эти имена синонимичны.)

Стирание фактически удаляет имя файла. Если это является только именем файла, то файл удален также. Если файл имеет другие имена также (см. Раздел 9.3 [Жесткие обращения]), то он остается доступным под другими именами.

### -Функция: int unlink (const char *filename)

Функция unlink удаляет filename имя файла. Если это является единственным именем файла, файл непосредственно также удален. (Фактически, если любой процесс работает с файлом, когда это случается, стирание откладывается, пока все процессы не закрыли файл.)

Unlink функция объявлена в заглавном файле 'unistd.h'.

Эта функция возвращает 0 при успешном завершении, и -1 при ошибке. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Имя файла Errors]), следующие errno условия ошибки определены для этой функции:

**EACCESS**

право записи отклонено для каталога из которого файл должен быть удален.

**EBUSY**

Эта ошибка указывает, что файл используется системой таким способом, что он не может быть удален.

**ENOENT**

имя файла, которое будет удалено не существует.

**EPERM**

На некоторых системах, unlink не может использоваться, чтобы удалить имя каталога, или может использоваться только привилегированным пользователем. Чтобы избегать таких проблем, используйте rmdir, чтобы удалить каталоги.

**EROFS**

Каталог, в котором имя файла должно быть удалено, находится в файловой системе только для чтения, и не может изменяться.

### -Функция: int remove (const char *filename)

Функция remove - другое имя для unlink. Remove - имя ANSI C, в то время как unlink - POSIX.1 имя. Remove имя объявлено в 'stdio.h'.

### -Функция: int rmdir (const char *filename)

Rmdir функция удаляет каталог. Каталог должен быть пуст прежде, чем он может быть удален; другими словами, он может только содержать элементы '. ' и ' .. '.

В большинстве других отношений, rmdir ведет себя подобно unlink. Имеются два дополнительных errno условия ошибки, определенные для rmdir:

**EEXIST**

**ENOTEMPTY**

Каталог, который будет удален не пуст.

Эти два кода ошибки синонимичны; некоторые системы используют один, а некоторые - другой.

Прототип для этой функции объявлен в заглавном файле 'unistd.h'.

# 4.6 Переименование Файлов

Функция rename используется, чтобы заменить имя файла.

```
int rename (const char *oldname, const char *newname) (функция)
```

Функция rename переименовывает имя файла oldname в newname. Файл, прежде доступный под именем oldname позже доступен как newname. (Если файл имел любые другие имена кроме oldname, он продолжает иметь те имена.)

Каталог, содержащий имя newname должен быть в той же самой файловой системе как файл (что обозначен именем oldname).

Один частный случай для rename - то, когда oldname и newname  два имени для того же самого файла. Непротиворечивый способ обрабатывать этот случай состоит в том, чтобы удалить oldname. Однако, POSIX говорит, что rename в этом случае не делает ничего, и не сообщает об ошибке. Мы не знаем то, что ваша операционная система будет делать. Система GNU будет возможно делать правильно (удалять oldname) если Вы явно не запрашиваете строгую POSIX совместимость 'даже когда она причиняет вред'.

Если oldname - не каталог, то любой существующий файл, именованный newname, удален в течение операции переименования. Однако, если newname - имя каталога, происходит сбой rename в этом случае.

Если oldname является каталогом, то или newname, не должен существовать, или должен быть указан каталог, который является пустым. В последнем случае, существующий каталог, именованный newname удален сначала. Имя newname не должно определить подкаталог каталога oldname, который переименовывается.

Если rename терпит неудачу, она возвращает -1. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Имя файла Errors]), следующие errno условия ошибки определены для этой функции:

**EACCES**

Один из каталогов, содержащих newname или oldname отказывает в записи; или newname и oldname - каталоги, и в праве записи отказано для одного из них.

**EBUSY**

Каталог, именованный oldname или newname используется системой способом, который предотвращает переименование во время работы. Это включает каталоги, которые являются точками крепления для filesystems, и каталогов, которые являются текущими рабочими каталогами процессов.

**EEXIST**

каталог newname существует.

### ENOTEMPTY

Каталог newname не пуст.

### EINVAL

oldname - каталог, который содержит newname.

### EISDIR

newname называет каталог, а oldname нет.

### EMLINK

Каталог предыдущего уровня newname имел бы слишком много связей.

Хорошо разработанные файловые системы никогда не сообщают эту ошибку, потому что они разрешают большое количество связей чем ваш диск, мог бы содержать. Однако, Вы должны все еще принимать во внимание возможности этой ошибки, поскольку она могла бы следовать из доступа к сети к файловой системе на другой машине.

### ENOENT

файл, именованный oldname не существует.

### ENOSPC

каталог, который содержал бы, newname не имеет никакого участка памяти для другого входа, и нет никакого места, оставшегося в файловой системе, чтобы расширить его.

### EROFS

Операция включила бы запись в каталогу на файловой системе только для чтения.

### EXDEV

Два имени файла newname и oldnames находятся в различных файловых системах.

## 4.7 Создание Каталогов

Каталоги создаются функцией mkdir. (Имеется также команда оболочки mkdir, которая делает то же самое.)

```
-Функция:     int mkdir (const char *filename, mode_t mode)
```

Mkdir функция создает новый, пустой каталог, с именем filename.

Mode аргумент определяет права файла для нового файла каталога. См. Раздел 9.8.5 [Биты Прав], для получения более подробной информации об этом.

Возвращаемое значение 0 указывает на успешное завершение, а -1 указывает на отказ. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:

**EACCES**

право записи отклонено для директории предыдущего уровня, в которую новый каталог должен быть добавлен.

**EEXIST**

файл, именованный filename уже существует.

**EMLINK**

директория предыдущего уровня имеет слишком много связей.

Хорошо разработанные файловые системы никогда не сообщают эту ошибку, потому что они разрешают большое количество связей чем ваш диск мог бы содержать. Однако, Вы должны все еще принимать во внимание возможности этой ошибки, поскольку она могла бы следовать из доступа к сети к файловой системе на другой машине.

**ENOSPC**

файловая система не имеет достаточного участка памяти, чтобы создать новый каталог.

**EROFS**

Директория предыдущего уровня создаваемого каталога находится в файловой системе только для чтения, и не может изменяться.

Чтобы использовать эту функцию, ваша программа должна включить заглавный файл 'sys/stat.h'.

# 4.8 Атрибуты Файла

Когда Вы применяете 'ls -l' команду оболочки на файле, это дает Вам информацию относительно размера файла, кто его хозяин, когда было последнее изменение, и т.п.. Этот вид информации называется атрибутами файла; они связаны с файлом непосредственно и не часть одного из имен.

Этот раздел содержит информацию относительно того, как Вы можете запрашивать и изменять эти атрибуты файлов.

## Что Означают Атрибуты Файла

Когда Вы читаете атрибуты файла, они возвращаются в структуре, называемой struct stat. Этот раздел описывает имена атрибутов, их типов данных, и что они означают.

Заглавный файл 'sys/stat.h' объявляет все символы, определенные в этом разделе.

```
struct stat         (тип данных)
```

Тип структуры stat используется, чтобы возвратить информацию относительно атрибутов файла. Она содержит по крайней мере следующие элементы:

```
mode_t st_mode
```

Определяет режим файла. Включая информацию о типе файла (см. Раздел 9.8.3 [Тестирование Типа Файла]) и биты прав файла (см. Раздел 9.8.5 [Биты Права]).

```
ino_t st_ino
```

Серийный номер файла, который отличает этот файл от всех других файлов на том же самом устройстве.

```
dev_t st_dev
```

Идентифицирует устройство, содержащее файл. St_ino и st_dev, вместе, однозначно идентифицируют файл.

```
nlink_t st_nlink
```

Число жестких связей с файлом. Этот счетчик следит, сколько каталогов имеют входы для этого файла. Если счетчик когда-либо уменьшится до нуля, то файл непосредственно отбрасывается. Символические связи не рассчитываются.

```
uid_t st_uid
```

ID владельца файла. См. Раздел 9.8.4 [Владелец Файла].

```
gid_t st_gid
```

ID группы файла. См. Раздел 9.8.4 [Владелец Файла].

```
off_t st_size
```

Это определяет размер файла в байтах. Для файлов, которые являются устройствами и т.п. это поле не значимо.

```
time_t st_atime
```

Это - время последнего доступа к файлу. См. Раздел 9.8.9 [Времена Файла].

```
unsigned long int st_atime_usec
```

Это - дробная часть времени последнего доступа к файлу.

```
time_t st_mtime
```

Это - время последней модификации содержимого файла.

```
unsigned long int st_mtime_usec
```

Это - дробная часть времени последней модификации содержимого файла.

```
time_t st_ctime
```

Это - время последней модификации атрибутов файла. См. Раздел 9.8.9 [Времена Файла].

```
unsigned long int st_ctime_usec
```

Это - дробная часть времени последней модификации атрибутов файла.

```
unsigned int st_nblocks
```

Это - количество дискового пространства, которое файл занимает, измеряемое в модулях (512-байтовых блоках).

Число блоков диска не строго пропорционально размеру файла, по двум причинам: файловая система может использовать, некоторые блоки для внутреннего хранения записи; и файл может быть разреженен, т. е. может иметь 'отверстия', которые содержат нули, но фактически не занимают пространство на диске.

Вы можете узнать (приблизительно) является ли файл разреженен, сравнивая это значение с st_size, примерно так:

```
(st.st_blocks * 512 < st.st_size)
```

Этот тест не совершенен, потому что файл, который только немного разрежен, не мог бы быть обнаружен как разреженный вообще. Для практических приложений, это - не проблема.

```
unsigned int st_blksize
```

Оптимальный размер блока для чтения или записи в этот файл. Вы могли бы использовать этот размер для распределения пространства буфера для чтения или для записи в файл.

Некоторые из атрибутов файла имеют специальные имена типа данных, которые существуют специально для этих атрибутов. (Они  все побочные результаты исследования для общеизвестных типов integer, которые Вы знаете и любите.) Эти typedef-имена определены в заглавном файле 'sys/types.h' также как в 'sys/stat.h'. Вот их список:

```
mode_t     (тип данных)
```

Это - тип данных integer, используемый, чтобы представить режимы файла. В системе GNU, это эквивалентно unsigned int.

```
ino_t      (тип данных)
```

Это - арифметический тип данных, используемый, чтобы представить серийные номера файла. (В UNIX жаргоне они иногда называются inode числами.) В системе GNU, этот тип эквивалентен long unsigned int.

```
dev_t      (тип данных)
```

Это - арифметический тип данных, используемый, чтобы представить числа файлового устройства. В системе GNU, это эквивалентно int.

```
nlink_t    (тип данных)
```

Это - арифметический тип данных, используемый, чтобы представить число связей файла. В системе GNU, это эквивалентно short unsigned int.

## Чтение Атрибутов Файла

Чтобы исследовать атрибуты файлов, используйте функции stat, fstat и lstat. Они возвращают информацию атрибута в объекте struct stat. Все три функции объявлены в заглавном файле 'sys/stat.h'.

```
int stat (const char *filename, struct stat *buf )  (функция)
```

Stat функция возвращает информацию относительно атрибутов файла, именованного filename в структуре, указанной в buf.

Если filename - имя символической связи, атрибуты, которые Вы получаете, описывают файл, на который ссылка указывает. Если ссылка направляет на несуществующее имя файла, то stat сбоит, сообщая несуществующий файл.

Возвращаемое значение - 0, если операция успешна, и -1 при отказе. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла],

следующие errno условия ошибки определены для этой функции:

**ENOENT**

файл, именованный filename не существует.

```
int fstat (int filedes, struct stat *buf )  (функция)
```

Fstat функция - подобна stat, за исключением того, что она берет открытый дескриптор файла как аргумент, вместо имени файла. См. Главу 8 [ввод - вывод низкого уровня].

Подобно stat, fstat возвращает 0 при успехе и -1 при отказе. Следующие errno условия ошибки определены для fstat:

**EBADF**

filedes аргумент - не допустимый дескриптор файла.

```
int lstat (const char *filename, struct stat *buf )  (функция)
```

Lstat функция - подобна stat, за исключением того, что она не следует за символическими связями. Если filename - имя символической связи, lstat возвращает информацию относительно связи непосредственно; иначе, lstat работает подобно stat. См. Раздел 9.4 [Символические Связи].

## Определение Типа Файла

Режим файла, сохраненный в st_mode поле атрибутов файла, содержит два вида информации: код типа файла, и биты прав доступа. Этот раздел обсуждает только код типа, который Вы можете использовать, чтобы сообщить является ли файл каталогом, является ли он гнездом, и так далее. Для уточнения информации относительно права доступа см. Раздел 9.8.5 [Биты Прав].

Имеются два предопределенных способа, которыми Вы можете обращаться к типу файла. Прежде всего для каждого типа файла, имеется макрокоманда предиката, которая исследует значение режима файла и возвращает истину или ложь если файл того типа, или нет. Во-вторых, Вы можете маскировать снаружи остальную часть режима файла, чтобы получить только код типа файла. Вы можете сравнивать его с различными константами типов файлов.

Все символы, перечисленные в этом разделе определены в заглавном файле 'sys/stat.h'.

Следующие макрокоманды предиката проверяют тип файла, заданный значением m, которое является st_mode полем, возвращенным stat на этом файле:

```
int S_ISDIR (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - каталог.

```
int S_ISCHR (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - символьный специальный файл (устройство подобное терминалу).

```
int S_ISBLK (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - блокированный специальный файл (устройство подобное диску).

```
int S_ISREG (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - регулярный(правильный) файл.

```
int S_ISFIFO (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - FIFO специальный файл, или канал. См. Главу 10 [Каналы и FIFO].

```
int S_ISLNK (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - символическая ссылка. См. Раздел 9.4 [Символические Связи].

```
int S_ISSOCK (mode_t m)  (макрос)
```

Эта макрокоманда возвращает отличное от нуля значение, если файл - гнездо. См. Главу 11 [Гнезда].

Альтернативный, не-POSIX метод тестирования типа файла обеспечивается для совместимости с BSD.

Режим можно поразрядно AND с S_IFMT, чтобы извлечь лод типа файл, и сравнить с соответствующей константой кода типа. Например,

```
S_ISCHR (mode)
```

эквивалентно:

```
((mode & S_IFMT) == S_IFCHR)
```

```
int S_IFMT  (макрос)
```

Это - битовая маска, используемая, чтобы извлечь код типа файла.

Вот символические имена для различных типов файлов:

```
S_IFDIR Эта макрокоманда представляет значение кода типа файла для файла
каталога.
S_IFCHR Эта макрокоманда представляет значение кода типа файла для файла -
устройства с символьной организацией.
S_IFBLK Эта макрокоманда представляет значение кода типа файла для блочно-
ориентированного файла.
S_IFREG Эта макрокоманда представляет значение кода типа файла для
регулярного(правильного) файла.
S_IFLNK Эта макрокоманда представляет значение кода типа файла для
символической связи.
S_IFSOCK Эта макрокоманда представляет значение кода типа файла для гнезда.
S_IFIFO Эта макрокоманда представляет значение кода типа файла для FIFO или
канала.
```

## Владелец Файла

Каждый файл имеет владельца, который является одним из зарегистрированных имен пользователей, определенных в системе. Каждый файл также имеет группу, которая является одной из определенных групп. Владелец файла может часто быть полезен, но его основная цель - управление доступом.

Владелец файла и группа играет роль в определении доступа, потому что файл имеет набор битов права доступа для пользователя, который является владельцем, другой набор, для

тех, кто принадлежат группе владельца файла, и третий набор битов, которые относятся ко всем остальным. См. Раздел 9.8.6 [Право Доступа]

Когда файл создан, его владелец определяется из пользовательского ID процесса, который создает его.

Вы можете изменять владельца и/или группу владельца существующего файла, используя chown функцию. Это - примитив для chown и chgrp команд оболочки.

Прототип для этой функции объявлен в 'unistd.h'.

```
int chown (const char *filename, uid_t owner, gid_t group) (функция)
```

Chown функция изменяет владельца filename файла, и группу.

Изменение владельца файла на некоторых системах очищает ID пользователя и ID-группы биты прав файла. (Потому что эти биты не могут соответствовать новому владельцу.) другие биты прав файла не изменяются.

Возвращаемое значение - 0 при успехе и -1 при отказе. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:

**EPERM**

Этот процесс испытывает недостаток прав, чтобы делать запрошенное изменение.

Только привилегированные пользователи или владелец файла могут изменять группу файла. На большинстве файловых систем, только привилегированные пользователи могут изменять владельца файла; некоторые файловые системы позволяют Вам изменять владельца, если Вы - в настоящее время владелец. Когда Вы обращаетесь к отдаленной файловой системе, поведение, с которым Вы сталкиваетесь, определено системой, которая фактически содержит файл, а не системой, на которой ваша программа выполняется.

См. Раздел 27.7 [Опции для Файлов], для уточнения информации относительно _POSIX_CHOWN_RESTRICTED макрокоманды.

**EROFS**

Файл находится в файловой системе только для чтения.

```
int fchown (int filedes, int owner, int group)  (функция)
```

Подобна chown, за исключением того, что она изменяет владельца файла на описателе файла filedes.

Возвращаемое значение из fchown - 0 при успехе и -1 при отказе. Следующие errno коды ошибки определены для этой функции:

**EBADF**

filedes аргумент - не допустимый дескриптор файла.

**EINVAL**

filedes аргумент соответствует каналу или гнезду, а не обычному файлу.

**EPERM**

Этот процесс испытывает недостаток прав, чтобы делать запрошенное изменение. Для подробностей, см. chmod, выше.

### EROFS

Файл постоянно находится в файловой системе только для чтения.

## Биты Режима для Прав Доступа

Этот раздел обсуждает биты права доступа, которые управляют чтением и записью в файл. См. Раздел 9.8.3 [Проверка Типа Файла], для уточнения информации относительно кода типа файла.

Все символы, перечисленные в этом разделе определены в файле 'sys/stat.h'.

Эти символические константы определены для битов режима файла, которые управляют правом доступа для файла:

### S_IRUSR

### S_IREAD

бит права чтения для владельца файла. На многих системах, этот бит - 0400. S_IREAD - устаревший синоним, предусмотрен для BSD совместимости.

### S_IWUSR

### S_IWRITE

бит права записи для владельца файла. Обычно 0200. S_IWRITE - устаревший синоним, предусмотрен для совместимости с BSD.

### S_IXUSR

### S_IEXEC

бит права записи для владельца файла. Обычно 0100. S_IWRITE - устаревший синоним, предусмотрен для совместимости cBSD.

### S_IRWXU

Это эквивалент ' (S_IRUSR | S_IWUSR | S_IXUSR) '.

### S_IRGRP

бит права чтения для владельца группы файла. Обычно 040.

### S_IWGRP

бит права записи для владельца группы файла. Обычно 020.

### S_IXGRP

бит права выполнения или поиска для владельца группы файла. Обычно 010.

### S_IRWXG

Это эквивалент ' (S_IRGRP | S_IWGRP | S_IXGRP) '.

### S_IROTH

бит права чтения для других пользователей. Обычно 04.

### S_IWOTH

бит права записи для других пользователей. Обычно 02.

### S_IXOTH

бит права выполнения или поиска для других пользователей. Обычно 01.

### S_IRWXO

Это эквивалент ' (S_IROTH | S_IWOTH | S_IXOTH) '.

### S_ISUID

бит выполнения, устанавливающий ID-пользователя, обычно 04000.

### S_ISGID

бит выполнения, устанавливающий ID-группы, обычно 02000.

### S_ISVTX

Это - <sticky> бит, обычно 01000.

На исполняемом файле, он изменяет политику подкачки системы. Обычно, когда программа завершается, страницы в ядре немедленно освобождены и многократно используются. Если sticky бит установлен на исполняемом файле, система, хранит страницы в ядре некоторое время, как будто программа все еще выполняется. Это выгодно для программы, которая должна быть выполнена много раз последовательно.

На каталоге, sticky бит дает право удалить файл в каталоге, если Вы можете записывать в содержимое этого файла. Обычно, пользователь либо может удалять все файлы в каталоге либо не может удалять никакой из них (имеет ли пользователь право записи для каталога). Липкий бит делает возможным управлять стиранием для индивидуальных файлов.

Фактические битовые значения символов перечислены в таблице выше, так что Вы можете декодировать значения режима файла при отладке ваших программ. Эти битовые значения правильны для большинства систем, но это не гарантируется.

Предупреждение: Запись явных чисел для прав файла - плохая практика. Это не только непереносимо, но также требует от каждого, кто читает вашу программу, помнить то, что

означают конкретные биты. Чтобы сделать вашу программу понятной, используйте символические имена.

## Как Разрешается Доступ к Файлу

Операционная система обычно разрешает право доступа к файлу, основываясь на ID пользователя и группы процесса, и на дополнительном ID группы, вместе с битами владельца, группы и битами права файла. Эти понятия обсуждены подробно в Разделе 25.2 [Свойства процесса].

Если ID пользователя процесса соответствует ID владельца файла, то права для чтения, записи, и выполнения/поиска, управляются соответствующими 'пользовательскими' (или 'владельца') битами. Аналогично, если любой из ID группы или дополнительной группы процесса соответствует ID группы владельца файла, то права, управляется битами 'группы'. Иначе, права управляются 'другими' битами.

Привилегированные пользователи, подобно 'root', могут обращаться к любому файлу, независимо от битов права файла. Как частный случай, для выполнения файла даже для привилегированного пользователя, по крайней мере один из битов выполнения должен быть установлен.

## Назначение Прав Файла

Примитивные функции для создания файлов (например, open или mkdir) воспринимают аргумент mode, который определяет права файла. Но заданный mode изменяется маской создания файла процесса, или перед использованием.

Биты, которые установлены в маске создания файла, идентифицируют права, которые должны всегда быть заблокированы для новых файлов. Например, если Вы устанавливаете все 'другие' биты доступа в маске, то новые файлы не доступны вообще для процессов в 'другом' классе, даже если аргумент mode, заданный в функции создания разрешил такой доступ. Другими словами, маска создания файла - дополнение обычных прав доступа, которые Вы хотите предоставить.

Программы которые создают файлы, обычно определяют аргумент mode, который включает все права, которые имеют смысл для специфического файла. Для обычного файла, это обычно право чтения и право записи для всех классов пользователей. Эти права ограничены как определено собственной маской создания файла индивидуального пользователя.

Чтобы изменять право существующего данного файла, вызовите chmod. Эта функция игнорирует маску создания файла; она использует только заданные биты права.

При нормальном использовании, маска создания файла инициализируется при входе пользователя в систему (использованием umask команды оболочки), и наследуется всеми подпроцессами. Прикладные программы обычно не должны заботиться о маске создания файла.

Когда ваша программа должна создать файл и обходить umask для прав доступа, самый простой способ сделать это состоит в том, чтобы использовать fchmod после открытия файла, а не изменять umask.

Фактически, изменение umask обычно делается только оболочками. Они используют umask функцию.

Функции в этом разделе объявлены в 'sys/stat.h'. mode_t umask (mode_t mask) (функция) Umask функция устанавливает маску создания файла текущего процесса, и возвращает предыдущее значение маски создания файла.

Вот пример, показывающий, как читать маску с umask без ее замены:

```
mode_t
read_umask (void)
{
        mask = umask (0);
        umask (mask);
}
```

```
mode_t getumask (void)  (функция)
```

Возвращает текущее значение маски создания файла для текущего процесса. Эта функция - расширение GNU.

```
int chmod (const char *filename, mode_t mode)  (функция)
```

Chmod функция устанавливает биты права доступа для файла, именованного filename как mode.

Если filename называет символическую ссылка, chmod изменяет право файла, указанного ссылкаю, а не связи непосредственно. Не имеется фактически никакого способа установить mode связи, который всегда  1.

Эта функция возвращает 0 в случае успеха и -1 если нет. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:

**ENOENT**

Именованный файл не существует.

**EPERM**

Этот процесс не имеет права изменить право доступа этого файла. Только владелец файла или привилегированный пользователь может изменять их.

**EROFS**

Файл постоянно находится в файловой системе только для чтения.

```
int fchmod (int filedes, int mode)  (функция)
```

Подобна chmod, за исключением того, что она изменяет права файла в настоящее время открытого через дескриптор filedes.

Возвращаемое значение из fchmod - 0 при успехе и -1 при отказе. Следующие errno коды ошибки определены для этой функции:

**EBADF**

filedes аргумент - не допустимый дескриптор файла.

**EINVAL**

filedes аргумент соответствует каналу или гнезду, или кое-чему еще, которое не имеет права доступа.

**EPERM**

Этот процесс не имеет права изменить право доступа этого файла. Только владелец файла или привилегированный пользователь может изменять их.

### EROFS

Файл постоянно находится в файловой системе только для чтения.

## Тестирование Прав для Обращения к Файлу

Когда программа выполняется привилегированным пользователем, это разрешает ей обращаться к файлам без ограничений на пример, изменять ' /etc/passwd '. Программы разработанные, чтобы быть выполненным обычными пользователями, но обращаться к таким файлам используют setuid так, чтобы они всегда выполнялись под root'ом.

Программа должна явно проверить, имел ли пользователь необходимый доступ к файлу, прежде, чем она начнет читать или писать в файл.

Для этого используйте функции доступа, которые проверяет право доступа, основанное на ID пользователя. (setuid не изменяет реального ID пользователя, так что это отражает пользователя, который фактически выполнил программу.)

Имеется другой способ, которым Вы могли бы проверить доступ, который является простым для описания, но очень интенсивно используемым. Нужно исследовать биты mode файла и подражать вычислению доступа системы. Этот метод нежелателен, потому что много систем имеют дополнительные возможности управления доступом; ваша программа не может корректно подражать им. Использование access просто и автоматически делает то, что соответствует системе, которую Вы используете.

Символы в этом разделе объявлены в ' unistd.h '.

```
int access (const char *filename, int how)  (функция)
```

Функция выясняет, можно ли к файлу, именованному filename обращаться способом, заданным аргументом how. Этот аргумент может быть либо поразрядным ИЛИ флагов R_OK, W_OK, X_OK, либо проверка существования F_OK.

Эта функция использует ID пользователя и группы процесса, а не эффективный ID, для проверки права доступа. В результате, если Вы используете функцию из программы setuid или setgid (см. Раздел 25.4 [Как Изменить Права]), это дает информацию относительно пользователя, кто фактически выполнил программу.

Возвращаемое значение - 0, если доступ разрешается, и -1 иначе. (Другими словами, обрабатываемая как функция предиката, access возвращает истину, если запрошенный доступ отклонен.)

В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени файла]), следующие errno условия ошибки определены для этой функции:

### EACCES

доступ, заданный how, отклонен.

### ENOENT

файл не существует.

**EROFS**

Право записи было запрошено для файла в файловой системе только для чтения.

Эти макрокоманды определены в заглавном файле 'unistd.h' для использования как аргументы функции access. Значения  константы integer.

```
int R_OK  (макрос)
```

Аргумент проверки на право чтения.

```
int W_OK  (макрос)
```

Аргумент проверки на право записи.

```
int X_OK  (макрос)
```

Аргумент проверки направо выполнения/поиска.

```
int F_OK Macro
```

Аргумент проверки на существование файла.

## Временные Характеристики Файла

Каждый файл имеет три временных метки, связанные с ним: время доступа, время изменения, и время изменения атрибута. Они соответствуют st_atime, st_mtime, и st_ctime элементам структуры stat; см. Раздел 9.8 [Атрибуты Файла].

Все эти времена представляются в календарном формате времени, как объекты time_t. Этот тип данных определен в 'time.h'. Для получения более подробной информации и манипулирования значениями времени см. Раздел 17.2 [Календарное Время].

Когда существующий файл открыт, его атрибуты, такие как время изменения, модифицируются. Чтение из файла модифицирует атрибут времени доступа, а запись модифицирует время изменения.

Когда файл создан, все три временных метки для этого файла установлены на текущее время. Кроме того, атрибут времени изменения каталога, который содержит новый вход, модифицируются.

Добавление нового имени для файла с функцией связи модифицирует атрибут поля времени изменения связываемого файла, и соответствующие атрибуты каталога, содержащего новое имя. Те же самые поля изменяются если имя файла удалено с unlink, remove, или rmdir. Переименование файла с rename воздействует только на атрибут времени изменения и изменения поля времени двух родительских включающих каталогов, а не на времена переименовываемого файла.

Изменение атрибутов файла (например, с chmod) модифицирует атрибут времени изменения.

Вы можете также изменять некоторые из временных меток файла, явно используются utime, за исключением изменения атрибута времени изменения. Вы должны включить заглавный файл 'utime.h' чтобы использовать это средство.

```
struct utimbuf  (тип данных)
```

Структура utimbuf используется с функцией utime, чтобы определить новый доступ или изменить времена для файла. Она содержит следующие элементы:

```
time_t actime
```

Это - время доступа(последнего) к файлу.

```
time_t modtime
```

Это - время изменения файла.

```
int utime (const char *filename, const struct utimbuf *times) (функция)
```

Эта функция используется, чтобы изменить файловые времена, связанные с файлом, именованным filename.

Если times являются пустым указателем, то время доступа и изменения файла устанавливаются на текущее время. Иначе, они устанавливаются как значения из actime и modtime элементов (соответственно) структуры utimbuf, указанной times. Utime функция возвращает 0 если обращение успешно и -1 при отказе. В дополнение к обычным синтаксическим ошибкам имени файла, следующие errno условия ошибки определены для этой функции:

**EACCES**

имеется проблема права в случае, где пустой указатель был передан как аргумент времена. Чтобы модифицировать временную метку на файле, Вы должны также быть владельцем файла и иметь право записи в файл, или быть привилегированным пользователем.

**ENOENT**

файл не существует.

**EPERM**

Если аргумент times - не пустой указатель, Вы должны также быть владельцем файла или привилегированным пользователем.

**EROFS**

Файл живет в файловой системе только для чтения.

Каждая из трех временных меток имеет соответствующую часть измеряющую микросекунды, которая расширяет разрешающую способность. Эти поля называются st_atime_usec, st_mtime_usec, и st_ctime_usec; каждое имеет значение от 0 до 999,999, которое указывает время в микросекундах. Они соответствуют tv_usec полю структуры timeval; см. Раздел 17.2.2 [Календарь с высоким разрешением].

Utimes функция - подобна utime, но также позволяет Вам определять дробную часть времен файла.

Прототип для этой функции находится в файле 'sys/time.h'.

```
int utimes (const char *filename, struct timeval tvp[2]) (функция)
```

Эта функция устанавливает доступ к файлу и времена изменения для файла, именованного filename.

Новый время доступа определено tvp [0], а новое время изменения tvp [1]. Эта функция происходит из BSD.

Возвращаемые значения и условия ошибки - такие же как для utime функции.

## 4.9 Создание Специальных Файлов

Mknod функция - примитив для создания специальных файлов, типа файлов, которые соответствуют устройствам. Библиотека GNU включает эту функцию для совместимости с BSD.

Прототип для mknod объявлен в 'sys/stat.h'.

```
int mknod (const char *filename, int mode, int dev)  (функция)
```

Mknod функция создает специальный файл с именем filename. Mode определяет режим файла, и может включать различные специальные биты файла, типа S_IFCHR (для символьного специального файла) или S_IFBLK (для блокированного специального файла). См. Раздел 9.8.3 [Тестирование Типа Файла].

Dev аргумент определяет, к которому устройству обращается специальный файл. Точная интерпретация зависит от вида создаваемого специального файла.

Возвращаемое значение - 0 при успехе и -1 при ошибке. В дополнение к обычным синтаксическим ошибкам имени файла (см. Раздел 6.2.3 [Ошибки Имени Файла]), следующие errno условия ошибки определены для этой функции:

**EPERM**

Процесс вызова не привилегирован. Только суперпользователь может создавать специальные файлы.

**ENOSPC**

каталог или файловая система, которая содержала бы, новый файл 'полна' и не может быть расширена.

**EROFS**

Каталог, содержащий новый файл не может изменяться, потому что он находится в файловой системе только для чтения.

**EEXIST**

уже имеется файл, именованный filename. Если Вы хотите заменить этот файл, Вы должны сначала удалить старый файл.

## 4.10 Временные Файлы

Если Вы должны использовать временный файл в вашей программе, Вы можете использовать tmpfile функцию для его открытия. Или Вы можете использовать tmpnam функцию, что бы сделать имя для временного файла и тогда открыть его обычным способом через fopen. Tempnam функция - подобна tmpnam, но допускает выбирать в какой каталог войдут временные файлы, и кое-что относительно того, на что их имена будут походить.

Эти средства объявлены в заглавном файле 'stdio.h'.

```
FILE * tmpfile (void)  (функция)
```

Эта функция создает временный двоичный файл для режима модификации, как будто, вызывая fopen с режимом ' wb + '. Файл удаляется автоматически, когда он закрыт или когда программа завершается. (На некоторых других системах ANSI C файл может не быть удаленным, если программа завершается неправильно).

```
char * tmpnam (char *result)  (функция)
```

Эта функция создает и возвращает имя файла, которое является допустимым именем файла, и не называет никакой существующий файл. Если аргумент result является пустым указателем, возвращаемое значение - указатель на внутреннюю статическую строку, которая могла бы изменяться последующими обращениями. Иначе, аргумент result должен быть указателем на массив по крайней мере из L_tmpnam символов, и результат будет написан в этот массив.

Возможно tmpnam будет терпеть неудачу, если Вы вызываете ее слишком много раз. Потому что фиксированная длина временного имени файла дает участок памяти для только конечного числа различных имен. Если tmpnam сбоит, она возвращает пустой указатель.

```
int L_tmpnam  (макрос)
```

Значение этой макрокоманды - константное выражение integer, которое представляет минимальный размер резервирования строки, достаточно большой, чтобы содержать имя файла, сгенерированное tmpnam функцией.

```
int TMP_MAX  (макрос)
```

Макрокоманда TMP_MAX - нижняя граница для того, сколько временных имен Вы можете создавать с tmpnam. Вы можете полагаться на способность вызвать tmpnam по крайней мере столько раз прежде, чем она будет терпеть неудачу, говоря что Вы сделали слишком много временных имен файла.

С библиотекой GNU, Вы можете создавать очень большое количество временных имен файла, если Вы фактически создаете файлы, возможно дисковое пространство закончится прежде, чем закончатся имена. Некоторые другие системы имеют фиксированное, малое ограничение числа временных файлов. Ограничение никогда не меньше чем 25. char * tempnam (const char *dir, const char *prefix) (функция)

Эта функция генерирует уникальное временное имя файла. Если prefix не пустой указатель, то до пяти символов этой строки используется как prefix для имени файла. Prefix каталога для временного имени файла определяется проверкой следующей последовательности. Каталог должен существовать и быть перезаписываемым.

- Переменная среды TMPDIR, если она определена.
- Аргумент dir, если это - не пустой указатель.
- Значение P_tmpdir макрокоманды.
- Каталог ' /tmp '.

Эта функция определена для SVID совместимости.

```
char * P_tmpdir (SVID макрос)
```

Эта макрокоманда - имя заданного по умолчанию каталога для временных файлов.

Более старые системы UNIX не имели таких функций. Взамен они использовали mktemp и mkstemp. Обе из этих функций работают изменяя строку шаблона имени файла, заданную Вами. Последние шесть символов этой строки должны быть ' XXXXXX '. Эти шесть X-ов заменятся на шесть символов, которые делают целую строку уникальным именем файла. Обычно строка шаблона - это что-нибудь вроде ' /tmp/prefixXXXXXX ', и каждая программа использует уникальный prefix.

Обратите внимание: Т. к. mktemp и mkstemp изменяют строку шаблона, Вы не должны передать строковые константы им. Строковые константы - обычно в памяти только для чтения, так что ваша программа разрушилась бы, если бы mktemp или mkstemp пробовали изменять строку.

```
char * mktemp (char *template)  (функция)
```

Mktemp функция генерирует уникальное имя файла, изменяя шаблон как описано выше. В случае успеха она возвращает модифицированный шаблон. Если mktemp не находит уникальное имя файла, она делает шаблон пустой строкой и возвращает ее. Если шаблон не заканчивается на ' XXXXXX ', mktemp возвращает пустой указатель.

```
int mkstemp (char *template)  (функция)
```

Mkstemp функция генерирует уникальное имя файла, точно так же как mktemp, но она также открывает файл для Вас через open (см. Раздел 8.1 [Открытие и Закрытие Файлов]). В случае успеха, она изменяет шаблон на месте и возвращает дескриптор файла открытый на этом файле для чтения и записи. Если mkstemp не может создать однозначно названный файл, она делает шаблон пустой строкой и возвращает -1. если шаблон не заканчивается на ' XXXXXX ', mkstemp возвращает -1 и не изменяет шаблон.

В отличие от mktemp, mkstemp, как гарантируют, создаст уникальный файл, который не может сталкиваться с любой другой программой, пробующей создать временный файл. Потому что она работает, вызывая open с O_EXCL битом флага, который говорит, что Вы хотите всегда создавать новый файл, и получать ошибку, если файл уже существует.